

# Report on Metaheuristic Optimization over String Theory Conjectures Model

Brandon Marquez Salazar

May 2026



# Chapter 1

## Work Context

This section regards the context of what was done until now. Before anything, the main topics involved in this project by the moment this report is written (*May 2026*) are *string theory* and *metaheuristic optimization*.

### 1.1 Theoretical Context

#### 1.1.1 Context on String Theory

Based on [1, 5].

String theory proposes that the fundamental elements of the universe aren't adimensional particles but rather one-dimensional strings of energy in a 10-dimensional space.

The mathematical framework of string theory shows us a way of understanding for the quantum gravity. This feature gave it the popularity it has today as a reliable search for the theory of everything.

Current version of string theory has 5 mathematical homologueous variants. This allows us to “change the window” with which study the strings behaviour.

The most relevant challenges in string theory are given by the swampland: *since the string theory is a framework it has the capacity to build a wide range of universes.*

Several conjectures and hypotheses about geometries where strings lives, string types, branes interaction, etc. are discussed currently, leading to a vast research field and to the creation of the swampland project, aimed to

find a way to distinguish theories inconsistent with quantum gravity from those that are consistent.

### 1.1.2 Context on Metaheuristic Optimization

Bioinspired algorithms are models inspired by the nature allowing us to understand and or predict the behaviour of complex systems [2]. Those algorithms are not necessarily strict-based on such phenomena but also inspired by them [4].

These algorithms are simple but their behaviour is complex, non-linear and adaptable, permitting solving problems involving similar complexity with ease [2].

However the effectiveness of any implementation is given by the problem, the algorithm's components definition and the model to optimize [2, 6].

Since there's a wide range of bio inspired algorithms aimed to solve complex and non-linear problems, the first and most important challenge is to find the one which fits better on the optimization. Also, to find the correct description of the phenomenon to optimize is crucial for a successful performance.

## 1.2 Objectives Until Now

The work is based on [3] approach implementing the same model, but now applying genetic algorithms.

The main goals of to this report are the following:

- Dissect and reverse engineering the main flow of the current implementation (wolfram notebook).
- Validate the new implementation of the model using a given database by Dr. Cesar Damián.
- Apply genetic algorithms to see the behaviour.
- Plot model solutions to see behaviour.
- Categorize solutions as stable or unstable (tachionic).
- Categorize potentials as dS (positive) or AdS (negative).

- Plot solutions potential vs. eigenvalues for visual categorization.

### 1.2.1 Wolfram Code Dissection

To achieve the reverse engineering of the wolfram code it was necessary to check firstly for the dataflow from the variables. Also, the paper [3] provided well defined mathematical expressions which helped to understand the symbolic elements of Mathematica.

The first analysed section was the following:

```
changmoduli = ({\[Tau] -> E^-\[Phi] vol^(1/2), \[Rho] -> vol^( 1/3)} /. {vol -> (\[Tau] Exp
  \[Phi]))^(3/2));
Print["VH3="]
VH3 = (AH3 Exp[2 \[Phi]])/\[Rho]^6 /. changmoduli /. {\[Phi] -> Log[1/s]}
Print["V03="]
V03 = A03 PowerExpand[(\[Rho]^((q - 6)/2) \[Tau]^~3) /. changmoduli] /. {\[Phi] -> Log[1/s]}
  /. {q -> 3}
Print["AF5="]
VF5 = AF5 PowerExpand[(\[Rho]^(3 - p) \[Tau]^~4) /. changmoduli] /. {\[Phi] -> Log[1/s]} /.
  {p -> 5}
VF5 = AF5/\[Tau]^4;
Print["VF3="]
VF3 = (AF3 Exp[4 \[Phi]])/\[Rho]^6 /. changmoduli /. {\[Phi] -> Log[1/s]}
Print["VD5="]
VR6 = AR6 PowerExpand[(\[Rho]^~1 \[Tau]^~2) /. changmoduli];
VD5 = AD5/(Sqrt[s] \[Tau]^(5/2))
Vins = AA Exp[-aa \[Tau]];
Print["Vreal = "]
Vreal = VH3 + VF3 + VD5 + VF5 + V03 + 0 Vins
var = {s, \[Tau]};
NM = Length[var];
(*
Potential descriptors section:
mi,j ..... is the hessian of the potential      /
detmi,j ..... is the determinant of the hessian  /
trmi,j ..... is the trace of the hessian          /
div ..... is the divergent of the potential       /
egval ..... stores the eigenvalues of the hessian  /
pfun ..... the function which likely evaluates    /
              the potential and extract the egvals /
              which later are used to return a value /
              $\lambda_{1,2}$ $\leq$ 0. This value is /
              the sum of both egvals or 0 for each /
              semidefinite positive egval          /
```

```

*)
Print["Hessian of the potential = "]
mij = Simplify[ Table[D[Vreal, var[[i]], var[[j]]], {i, 1, NM}, {j, 1, NM}]]
detmij = Det[mij];
trmij = Tr[mij];
diV = Simplify[Table[D[Vreal, var[[i]]], {i, 1, NM}]];
egval = Eigenvalues[ Table[D[Vreal, var[[i]], var[[j]]], {i, 1, 2}, {j, 1, 2}]];
pfun = Function[{s, \[Tau]}, 10^4 If[egval[[1]] < 0, egval[[1]], 0] + 10^4 If[egval[[2]] <
0, egval[[2]], 0]];

```

Which resulted in the following when implemented into python:

```

import sympy          as sp #type: ignore
##### Definición de las variables principales de la función potencial
AH3 = sp.Symbol('AH3', real=True)
AF3 = sp.Symbol('AF3', real=True)
AF5 = sp.Symbol('AF5', real=True)
A3N3 = sp.Symbol('A3N3', real=True)
AD5 = sp.Symbol('AD5', real=True)
tau = sp.Symbol('tau', real=True)
s = sp.Symbol('s', real=True)
##### Definición de las variables principales de la función potencial
##### Diccionario de valores numéricos globales
variables_numericas={
    "AH3" : 1.0,
    "AF3" : 1.0,
    "AF5" : 1.0,
    "A3N3": 1.0,
    "AD5" : 1.0,
    "tau" : 1.0,
    "s"   : 1.0
}
##### Diccionario de valores numéricos globales

# Función potencial V
V = (AH3*s/(tau**3)) + (AF3/(s*tau**3)) + (AF5/(tau**4)) + (A3N3/(tau**3))
V_lifting = (AH3*s/(tau**3)) + (AF3/(s*tau**3)) + (AF5/(tau**4)) + (A3N3/(tau**3)) + (AD5
/((s**sp.Rational(1,2)*(tau**sp.Rational(5,2)))))
#V_lifting = (AH3*s/(tau**3)) + (AF3/(s*tau**3)) + (AF5/(tau**4)) + (A3N3/(tau**3)) + (AD5
/((s**(1/2)*(tau**(5/2)))))
# Función potencial V

# Matriz Hessiana de la función potencial
VHess = sp.hessian(V, [s,tau])
V_liftingHess = sp.hessian(V_lifting, [s,tau])
# Matriz Hessiana de la función potencial

```

```
# Autovalores de VHess
eigenvals_VHess = VHess.eigenvals()
eigenvals_VHess_lifting = V_liftingHess.eigenvals()
# Autovalores de VHess
```

## Workflow Context

Rather than just showing the code the structure of the wolfram optimization implementation follows the flow below:





## Chapter 2

# Regarding Proposals and Hypotheses

Given the results previously shown, I permit myself to extend some conjectures and hypotheses to which I reached. In addition to offer some possible approaches to the problem.

### 2.1 Conjectures and Hypotheses

Given the nature of the model, the lack of boundaries didn't affect considerably the results. Which I think is part of what is called **inevitability**. Such inevitability is an interesting aspect from the computation point of view due to “natural falling” into the correct values to minima without the need of constraints.

### 2.2 Proposals

For a better minima search I propose to use the squared potential as part of the penalizations, since the behaviour at minimization is to always fall on the root (this case, the zero).